

SMS Project

Smart Machine Systems, summer 2026

This document is to introduce you into the technical details of the project task for this years SMS course.

Assumptions

1. You have access to a Windows/Mac/Linux computer, optimally with 16 GB+ memory (RAM)

Installation

In all cases, you will be prompted to select the memory, storage and CPU cores to make available to the virtual machine. We have had good results with 8192MB, 30GB, and >4 cores respectively. If your computer can't provide these resources, you can experiment with less memory and CPU cores, which is untested but should work (slower).

Windows:

1. Select the appropriate virtualization software:
 - If you are on Windows 10/11 Pro, try to use Hyper-V Manager. See [here](#) for installing Hyper-V Manager and [here](#) for installing a VM.
 - Disable Secure Boot in the virtual machine's settings if it does not boot
 - If this is not possible (because you are on a "Home" version of Windows), install Oracle VirtualBox (<https://www.virtualbox.org/>). This has a more complex setup process. An exemplary guide is <https://itslinuxfoss.com/install-ubuntu-22-04-virtualbox/>, with many more on the internet.
2. Install a virtual machine (VM) of Ubuntu 22.04 LTS (<https://releases.ubuntu.com/jammy/>)
3. Start the virtual machine and create an arbitrary user
 - User name is not relevant for us, please remember the password!
 - When installing, select the "Minimal Install" as you will not need all the optional software :)

⚠ Important Settings in VirtualBox

There currently are some pitfalls working with the VirtualBox virtual machine. These can lead to you not being able to open a terminal or start MuJoCo.

- When creating the virtual machine, uncheck "Unattended Installation", as this creates an unprivileged user.
- If you are on Windows 11, make sure Hyper-V is disabled.

Sometimes, certain features of your hardware do not get passed into the virtual machine. Check that the AVX instruction set is available by executing `grep avx /proc/cpuinfo` If nothing is shown, that means AVX is not available. This is needed by MuJoCo and is oftentimes caused by Hyper-V running somewhere in the background still. Otherwise, if there is any output, you should be fine. If problems persist, please come into the tutorial session.

MacOS (tested on Apple Silicon):

- Use the UTM virtualization software
- You can't directly install Ubuntu Desktop but need to install the full Desktop from the Server edition. This is detailed in this guide: <https://www.youtube.com/watch?v=1PL-0-5BNXs>
 - Instead of running `sudo apt install ubuntu-desktop`, you can try `sudo apt install --no-install-recommends ubuntu-desktop` for a less bloated version of Ubuntu (without, e.g., the full LibreOffice suite)

Inside the VM:

1. Open a terminal
2. Install git: `sudo apt install git`
 - Whenever you enter `sudo`, you are doing a privileged (admin) action. You will be prompted to enter the password of the account inside the VM. Typically, you can't see the characters you are entering, but they exist :)

3. Download our software package:

```
git clone https://github.com/ma4096/unitree_sdk2_docker.git
```

4. Install the software (execute line by line):

```
cd unitree_sdk2_docker
chmod +x ./install_mono.sh
source ./install_mono.sh
```

If you are on Linux natively, follow the steps outlined in the README.md of the `unitree_sdk2_docker` repo (short installation). Usage is a bit different, but you can use `./start_[docker, mujoco]` to launch the SDK or MuJoCo containers respectively.

- Using `./install_mono` works too, but can make changes to your local system. If you want to use it anyway, consider sourcing a Python virtual environment before running the installation.

Orientation

The software consists of two main pieces:

- The software you write, supported by our Python libraries that are now installed.
- The simulation software (MuJoCo) that communicates with your software just like the real robot would do. For this simulation, you will build the simulation scene to resemble our lab.

The now installed folders should look like:

```
unitree_sdk2_docker/
├── code/
│   └── all your stuff should happen here :)
├── unitree_mujoco/
│   ├── simulate_python/
│   │   ├── config.py
│   │   └── unitree_mujoco.py
│   ├── unitree_robots/
│   │   ├── go2/
│   │   │   └── scene.xml
│   │   └── ...
│   └── ...
├── isem_go2_interface/
│   ├── src/
│   │   ├── isem_go2_interface/
│   │   │   ├── Go2.py
│   │   │   └── OnnxController.py
│   │   └── ...
│   └── ...
├── venv/
│   └── bin/
│       └── activate
└── (other folders not directly relevant to you)
```

Activating a virtual environment

The Python packages get installed into a virtual environment. This isolates your system environment from your development activities. To activate (“source”) the environment, execute

```
source venv/bin/activate
```

from the `unitree_sdk2_docker` directory. This must be done in every terminal you intend to use in this project.

Development usage

A normal usage in the scope of this project would be to have a code editor like VSCode open with the `unitree_sdk2_docker` folder and editing code inside the `code` directory or the `unitree_mujoco/unitree_robots/go2/` simulation scene.

In two terminals, you can run the simulation and your code simultaneously (see the next chapters).

Building the simulation

We use a modified simulation environment based on Unitree's official MuJoCo version (https://github.com/unitreerobotics/unitree_mujoco) which integrates more sensors and support for low performance hardware.



What is MuJoCo?

MuJoCo (Multi-Joint dynamics with Contact) is a simulation package for rigid robotics originally developed by Google. Like all robotic simulation tools, it takes inputs (like torque applied to a joint), simulates the physical reaction of the system, and returns some defined sensor readings. For us, it is also important that the system can be visually observed.

The actual python scripts for the simulation are placed inside the `unitree_sdk2_docker/unitree_mujoco/simulate_python` folder.

- `config.py` sets some options. If you have installed this software using `install_mono.sh`, make sure to change the `INTERFACE` to `lo` (line 4).
- `unitree_mujoco.py` runs the simulation.

To start the simulation, run

```
python unitree_mujoco.py
```

An empty world with only the robot and some boxes will show up. Look, zoom, and move around using the mouse and explore the settings to make yourself familiar with the interface.

This scene is defined in `unitree_sdk2_docker/unitree_mujoco/unitree_robots/go2/scene.xml`. Edit the boxes that are in the scene by changing some of their properties inside the `<worldbody>` tag. After saving the file and restarting the simulator, you can observe the changes.

Experiment with these properties, create new bodies, and look up how to include CAD models (we observed it to be easiest with `.obj` files).

Inside this `scene.xml` file, you will build the simulation environment in which the robot will move according to the code you write.

Programming

In the **code** directory, your code lives. When we deploy the software on the real robot, we will copy everything inside this directory.

An example structure for a Python file would be:

```
from isem_go2_interface import Go2
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from time import sleep

# Setup the communication with the robot
# This is the only thing that would need to be changed to switch between simulation
and the real robot#
# If installed using the `install_mono.sh` script, please use "lo" insted of "eth0"
ChannelFactoryInitialize(0, "eth0")

go2 = Go2()

# We assume a starting position where the robot can raise up safely
go2.standup()
print("Stood up")

# move forward with 0.5m/s and 0.5rad/s for 2 seconds
go2.move(0.5, 0, 0.5, 2)
print("Moved!")
# move backwards with 1m/s for 2 seconds
go2.move(-1, 0, 0, 2)

sleep(5) # the robot will stand for 5 seconds with a movement command of 0,0,0

go2.damp()
print("Damp!")
# this takes 5s, to stabilize before doing anything else.
```

Let's go through this code step by step:

- The first lines import all relevant libraries, classes and functions. `isem_go2_interface` is designed by us.
- `ChannelFactoryInitialize` sets up the communication between the robot/simulator and your code. `0` and `eth0` specify the channels: the first is the domain, the second specifies the network device to broadcast messages over.
- `go2 = Go2()` creates an object resembling the Go2 robot.
- `go2.standup()` lets the robot stand up from a laying down position. See Fig 1 and 2.
- `go2.move(...)` gives the robot a velocity command that it will follow for the specified amount of seconds. This is more of a rough estimate, as the following motion is generated by a machine learning algorithm.
- `go2.damp()` is used as an emergency stop function. This resembles the command used by Unitree in the remote control.

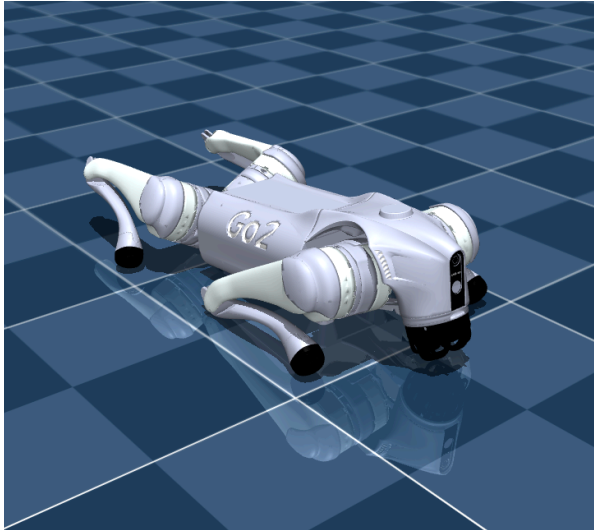


Figure 1: Laying down

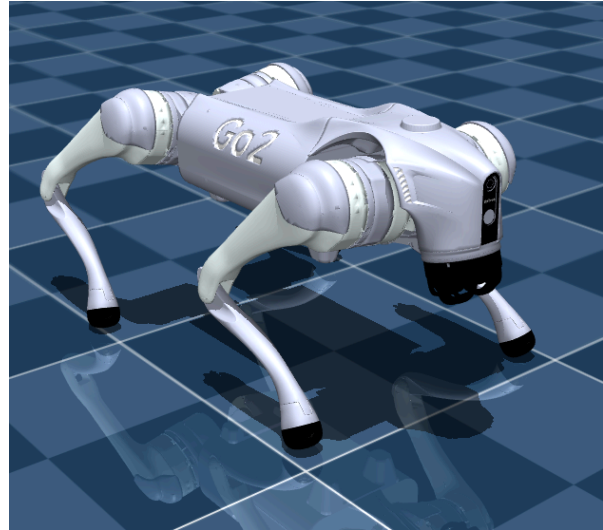


Figure 2: After standing up



Documentation tip

Inside VSCode, hovering over functions can show their docstring. This is a built in way to document software in python projects. We implement this for our functions inside the `isem_go2_interface`. For more information into docstrings, see for example https://www.sphinx-doc.org/en/master/usage/extensions/example_google.html.

With these functions (and others you find inside the `Go2.py` file), you should be able to fully control the movement of the robot.

Some Linux basic commands

To start a Python program, you need to call it from a terminal. We collected some frequently used commands to work with terminals in Linux.

- Move to a folder: `cd [folder_name]`
 - Return to the parent folder by calling `cd ..`
- List all files and folders in the current directory: `ls`
- Install a Python package: `pip install [package-name]`
 - Be cautious with installing packages that you don't know or were given to you by an LLM. Always search for them online prior to installing, and check if their GitHub repositories or pypi.org entries look trustworthy.
- Run a Python script: `python [filename].py`
 - In some installations of Python, only `python3` instead of `python` works.
- One program typically blocks the terminal.
 - Stop the program (if it does not end by itself) with `Ctrl+C`
 - Start another program while one is running by opening a new terminal
- Copy and paste in the terminal is done by `Ctrl+Shift+C` and `Ctrl+Shift+V` respectively.